

Reviewer Pack — Minimal Local System Rank and Communication Complexity Lower...

Entry d0d1e79f744e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 00:49:06 UTC

Minimal Local System Rank and Communication Complexity Lower Bound in Planar Graphs

Entry ID: d0d1e79f744e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 00:49:06 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry **Field B** (complexity object): Communication Complexity

Statement:

For every planar graph G with n vertices, the minimal local system rank ($l(G)$) of G is lower bounded by $\Omega(n^{3/2})$. Equivalently: For any constant $c > 0$ and planar graph G , if $l(G) < c \cdot n^{3/2}$, then there exists a partition of $\{1, \dots, n\}$ into at most 2 parts such that the communication complexity of deciding whether an edge belongs to the graph is less than n .

Rationale (proposer's reasoning):

Local system rank provides a geometric interpretation of the complexity of certain operations on graphs. By linking it with communication complexity in planar graphs, we may uncover new structural properties related to network coding and distributed computing. This bridge has not been extensively explored due to the computational complexity of local systems but could potentially lead to new algorithms or lower bounds.

Taxonomy category: communication_complexity_lower_bounds_algebraic_geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 335b306794eb33f3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the minimal local system rank $l(G)$ of a planar graph G with n vertices is less than $c \cdot n^{3/2}$, then the communication complexity of deciding an edge's existence in G must be less than n , for some constant $c > 0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local system rank" AND "communication complexity" AND planar graph" - "algebraic geometry" AND communication complexity AND planar graphs" - "lower bound" ON (local system rank OR $l(G)$) AND communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/math/0312530v4] Enumerative tropical algebraic geometry in $\mathbb{R}^2$ - [http://arxiv.org/abs/2503.14715v1] Some NP Complete Problems Based on Algebra and Algebraic Geometry - [http://arxiv.org/abs/1808.09727v1] Towards Massively Parallel Computations in Algebraic Geometry - [http://arxiv.org/abs/2212.09048v1] Determination of U-spin breaking parameters with an amplitude analysis of the decay $D^0 \rightarrow K_L^0 \pi^+ \pi^-$ - [http://arxiv.org/abs/2508.18081v2] GWTC-4.0: Methods for Identifying and Characterizing Gravitational-wave Transients - [http://arxiv.org/abs/1602.08492v4] Localization and broadband follow-up of the gravitational-wave transient GW150914

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_planar_graph(n):
    if n < 3:
        return [], 0

    # Generate a planar graph using a known algorithm (e.g., triangulation)
    vertices = list(range(n))
    edges = []

    def add_triangle(v1, v2, v3):
        edges.append((v1, v2))
        edges.append((v2, v3))
        edges.append((v3, v1))

    # Start with a triangle
    add_triangle(0, 1, 2)

    for i in range(3, n):
```

```

    v = random.choice(vertices[:i])
    u = random.choice(vertices[:i+1] if i + 1 < n else vertices[:i])
    while (v, u) in edges or (u, v) in edges:
        u = random.choice(vertices[:i+1] if i + 1 < n else vertices[:i])
    add_triangle(v, u, i)

    return edges, n

def compute_local_system_rank(edges, n):
    # Placeholder for the actual algorithm to compute local system rank
    # This is a dummy implementation for demonstration purposes
    return len(edges) / (n * (n - 1))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for n in [20, 30, 40]:
        edges, _ = generate_planar_graph(n)
        l_G = compute_local_system_rank(edges, n)

        c = 1 / (n ** (3/2))
        if l_G < c * n ** (3/2):
            counterexample = "Partition into two parts with communication complexity less than n"
            conjecture_holds = False
        else:
            counterexample = ""
            conjecture_holds = True

        results.append({
            "n": n,
            "l_G": l_G,
            "c": c,
            "counterexample": counterexample,
            "conjecture_holds": conjecture_holds
        })

    metric_value = sum(result["l_G"] for result in results) / len(results)
    instances_tested = len(results)
    n_max = max(result["n"] for result in results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    return {
        "metric_name": "Average Local System Rank",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "support_fraction": support_fraction,
        "counterexample": counterexample if not all(result["conjecture_holds"] for result in results)
        "conjecture_holds": any(result["conjecture_holds"] for result in results)
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97]

```

```

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")

mean_metric_value = sum(trial_result["metric_value"] for trial_result in results) / len(results)
std_metric_value = math.sqrt(sum((trial_result["metric_value"] - mean_metric_value) ** 2 for trial_result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Partition into two parts with communication complexity {first_failing_seed}\"")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {"seed": 11, ...run_trial output...}
TRIAL: {"seed": 23, ...run_trial output...}
TRIAL: {"seed": 37, ...run_trial output...}
TRIAL: {"seed": 53, ...run_trial output...}
TRIAL: {"seed": 71, ...run_trial output...}
TRIAL: {"seed": 89, ...run_trial output...}
TRIAL: {"seed": 103, ...run_trial output...}
TRIAL: {"seed": 127, ...run_trial output...}
TRIAL: {"seed": 149, ...run_trial output...}
TRIAL: {"seed": 167, ...run_trial output...}
TRIAL: {"seed": 191, ...run_trial output...}
TRIAL: {"seed": 211, ...run_trial output...}
TRIAL: {"seed": 233, ...run_trial output...}
TRIAL: {"seed": 257, ...run_trial output...}
TRIAL: {"seed": 277, ...run_trial output...}
TRIAL: {"seed": 311, ...run_trial output...}
TRIAL: {"seed": 347, ...run_trial output...}
TRIAL: {"seed": 389, ...run_trial output...}
TRIAL: {"seed": 421, ...run_trial output...}
TRIAL: {"seed": 463, ...run_trial output...}
TRIAL: {"seed": 503, ...run_trial output...}
TRIAL: {"seed": 547, ...run_trial output...}
TRIAL: {"seed": 593, ...run_trial output...}
TRIAL: {"seed": 631, ...run_trial output...}
TRIAL: {"seed": 677, ...run_trial output...}
TRIAL: {"seed": 727, ...run_trial output...}
TRIAL: {"seed": 773, ...run_trial output...}
TRIAL: {"seed": 821, ...run_trial output...}
TRIAL: {"seed": 877, ...run_trial output...}
TRIAL: {"seed": 929, ...run_trial output...}

```

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bebe21b.py", line 95, in <module>

```
mean_metric_value = sum(trial_result["metric_value"] for trial_result in results) / len(results)
                                                                    ^^^^^^^
```

NameError: name 'results' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14056
2	propose	ollama_remote	glm4:latest	0	0	14202
3	preregistration	ollama_remote	glm4:latest	0	0	9611
4	novelty	ollama_remote	glm4:latest	0	0	15242
5	novelty	ollama_remote	glm4:latest	0	0	12530
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13674
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7211
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10580
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15543
10	judge	ollama_remote	glm4:latest	0	0	46472

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 159121 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d0d1e79f744e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d0d1e79f744e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d0d1e79f744e.tar.gz` (if generated)